µOS++ III on SMP Boards

How the kernel, the ports and the boards run threads on several cores

Kernel: micro-os-plus-iii (SMP scheduler)

Ports: aarch32, aarch64, cortexm, posix-arch

Tree: smp (new-critical-sections merged)

SMP in plain English (start here)
- The one-paragraph version
- The kitchen analogy
- The five rules in everyday words
- What SMP is not
- The words, translated
- One story: a producer, a consumer, two cores

1. The short version
- How to read this document

2. What runs where
- 2.1 The layers
- 2.2 The compile-time fork

3. The configuration that makes an SMP build
- 3.1 Board facts, not defaults
- 3.2 The switch list
- 3.3 An example

4. Shared and per-CPU state
- 4.1 What the kernel keeps once
- 4.2 What is per-CPU
- 4.3 The thread object in SMP

5. Locks and critical sections
- 5.1 Why masking interrupts is not enough
- 5.2 The recursive kernel spinlock
- 5.3 Three critical sections, and what each protects
- 5.4 The rules that keep it correct
- 5.5 Memory ordering

6. Scheduling
- 6.1 The ready list and the picker
- 6.2 Affinity and idle threads
- 6.3 Preemption and time slicing
- 6.4 A switch requested under a lock
- 6.5 What happens when no thread is ready
- 6.6 Statistics

7. Context switching and the deferred publish
- 7.1 The hazard
- 7.2 The protocol
- 7.3 The saved frame, per port
- 7.4 The first switch

8. Booting the other cores
- 8.1 The sequence
- 8.2 How each board releases a core
- 8.3 The join handshake
- 8.4 The secondary's own state

9. Inter-processor interrupts
10. Time on multiple cores
11. Kernel objects and inter-thread communication
- 11.1 Waiting and waking
- 11.2 Mutexes and priority inheritance
- 11.3 Condition variables, semaphores, queues, flags
- 11.4 Memory and libc
- 11.5 errno and this_thread

12. Thread lifecycle on SMP
13. The ports compared
14. Porting to a new SMP board
- 14.1 Checklist
- 14.2 The three questions a new port must answer

15. Testing what was built
- 15.1 What each class of test proves
- 15.2 Running them
- 15.3 What a healthy boot looks like

16. Limits, costs and sharp edges
17. Source map and glossary
- 17.1 Where to look
- 17.2 Glossary
- 17.3 Related documents

18. What happens when...?
19. Common misunderstandings
20. Debug by symptom
21. One-page cheat sheet
- The five invariants
- The three locks
- Boot a secondary in six steps
- Thread states
- When an IPI is sent
- Numbers to remember

22. Read the source in this order

SMP in plain English (start here)

> This part avoids code names for the first few pages. If you already know what a core, a thread and a context switch are, skip to section 1.

The one-paragraph version

µOS++ III on an SMP board is one small operating system running on several processor cores at once. There is **one** list of threads that are ready to run, **one** scheduler, and **one** set of rules. Each core runs one thread at a time and remembers which thread it is running. Because the cores share memory, they need a way to take turns when they change shared data: a single "kernel lock" that only one core can hold at a time. When a core switches from one thread to another, it must finish saving the old thread before any other core is allowed to run it. That rule is the "claim and publish" protocol. Everything else in this document is bookkeeping around those ideas.

The kitchen analogy

Think of the board as one busy restaurant kitchen.

- The **cores** are cooks. Two, three or four of them work in the same kitchen.
- A **thread** is a recipe card: a list of instructions that someone wants cooked. The operating system does not know what the recipe means; it only knows how to hand it to a cook.
- The **ready list** is the rail where recipe cards wait for a free cook. Cards are ordered by importance (priority), and equally important cards take turns.
- Each cook holds one card at a time. The card in a cook's hand is that core's **current thread**.
- The **kernel lock** is the single key to the pantry. Only the cook holding the key may change the shared shelves: the order rail, the waiting lists, the message queues, the heap. Everyone else waits outside.
- **Masking interrupts** means "do not tap me on the shoulder for a moment." It stops that one cook from being interrupted. It does **not** stop the other cooks, which is exactly why the pantry key is needed.
- When a cook puts a half-finished dish down, they must step away from the stove before another cook can pick it up. In the kernel, a thread's saved context is put back on the "can run" shelf only after the core has left that thread's stack. That is **deferred publish**.
- An **IPI** is one cook tapping another on the shoulder: "look at the rail now." It is how one core asks another to reschedule immediately.
- A cook with nothing to do stands by the stove and waits for an order. That is the **idle thread**.
- A recipe may say which stoves it is allowed to use. That is **affinity**.

The analogy is not perfect, but it captures the important part: several workers, one set of shared shelves, one key, and a strict rule about when a half-finished dish may be touched.

The five rules in everyday words

1. **One queue, one key.** There is one list of runnable threads and one kernel lock. Any core that wants to change the list, a queue, a mutex or the heap takes the key first. *If you break it:* two cores edit the same list at once and a thread disappears, appears twice, or points at freed memory.
2. **Each cook holds one recipe, and the ticket says which.** Every core has its own current thread. A thread that is running on one core is not also sitting in the queue. *If you break it:* the same thread is resumed on two cores at once and both use the same stack.
3. **A recipe card goes back on the rail only when the cook's hands are off it.** A saved context becomes visible to other cores only after the CPU has left that thread's stack. Until then it is marked "not claimable." *If you break it:* another core starts running a thread whose registers are still being saved, and the two cores write the same stack.
4. **"Do not switch recipes on my stove" is not the same as "lock the pantry."** The scheduler lock stops this core from switching threads. It may or may not exclude other cores, depending on the port. The kernel lock is what protects shared data. *If you break it:* you "protect" a shared counter with the scheduler lock, and on AArch64 the other core changes it anyway.
5. **Never sit down to wait while holding the pantry key.** Kernel critical sections are short: update a pointer, unlink a node, release. Do not print, do not do I/O, do not block. *If you break it:* every other core spins with its interrupts masked until you finish, and the system looks frozen.

What SMP is not

- **It is not two operating systems.** There is one kernel, one thread list, one clock, one heap. The cores are workers inside the same system.
- **It is not automatic speed-up.** One thread still runs on one core. Only independent work can use more than one core at a time.
- **It is not a licence to share data carelessly.** The kernel protects its own data. Your data needs your own mutex, queue, atomic or explicit kernel section.
- **It is not "interrupts off everywhere."** Masking interrupts on core 0 has no effect on core 1. That is the whole reason the kernel lock exists.
- **It is not instant remote preemption.** A thread made ready on another core usually starts at that core's next tick, unless the kernel sends an IPI. One tick is 1 ms by default.
- **It is not a runtime mode.** SMP is chosen when the image is built. A single-core image and an SMP image are different binaries.

The words, translated

Word in the code	What it means in plain English	Kitchen picture
core, CPU	One processor that can execute instructions	One cook
thread	An independent sequence of instructions	A recipe card
scheduler	The code that decides which thread runs next	The head chef
ready list	The threads that could run now	The order rail
current thread	The thread a core is running	The card in a cook's hand
context switch	Saving one thread and starting another	Putting down one recipe and picking up another
kernel lock	The one lock that protects shared kernel data	The pantry key
scheduler lock	A lock that stops this core from switching threads	"Do not change my recipe yet"
interrupt mask	Tells this core's hardware "do not interrupt me now"	"Do not tap my shoulder"
spinlock	A lock whose waiter keeps asking until it is free	Waiting at the pantry door
IPI	An interrupt one core sends to another	A tap on the shoulder
tick	The periodic timer interrupt	The kitchen timer
idle thread	The thread a core runs when nothing is ready	A cook waiting for orders
affinity	The set of cores a thread may run on	Which stoves a cook may use
claim	Mark a saved thread "in use, do not pick it"	Taking the card off the rail
publish	Make a saved thread available to other cores	Putting the card back on the rail
mutex	A lock for application data, with blocking	A reservation on one shelf
semaphore	A counter that wakes waiters	A stack of order tokens
message queue	A bounded buffer with blocking send/receive	The pass-through window

One story: a producer, a consumer, two cores

Let us follow one concrete run. There are two worker threads: a **producer** on core 0 and a **consumer** on core 1. They share a message queue. For this story both threads are pinned to their cores, so the IPI path is visible.

1. **Both threads are created.** Each gets a stack, a saved context and a node on the ready list. They are not running yet; they are cards on the rail.
2. **Core 0 picks the producer, core 1 picks the consumer.** Each core removes its card from the rail, marks the thread "running," and stores it in its own `current_thread_[cpu]`. The threads are now in the cooks' hands.
3. **The consumer asks for a message and the queue is empty.** Inside the kernel lock, it puts a wait node on the queue's waiting list, marks itself "suspended," and releases the lock. Then it asks the scheduler to switch. Core 1 now runs its idle thread, because there is nothing else for it.
4. **The producer sends a message.** Inside the kernel lock, it copies the message into the queue's buffer and unlinks the consumer's wait node. It puts the consumer back on the ready list: the card goes back on the rail. Then it releases the lock.
5. **The producer asks the scheduler to switch.** On core 0, the producer may keep running if it still has the highest priority. But the consumer is pinned to core 1, and core 0 is not allowed to run it, so the kernel sends an IPI to core 1: "look at the rail now."
6. **Core 1 takes the IPI.** The interrupt handler acknowledges the interrupt, sets "a switch is pending" for core 1, and returns. On the way out of the handler, the port performs the switch.
7. **Core 1's scheduler scans the ready list.** The first eligible thread is the consumer. Its saved context is complete and visible, so core 1 **claims** it: it removes the card from the rail and marks the saved context "in use."
8. **Core 1 switches to the consumer.** It restores the consumer's registers and stack, and the consumer continues exactly after its `receive()` call. The message is there, so it processes it.
9. **The producer eventually exits.** It removes itself from the ready list and puts itself on the "terminated" list. It does not free its own stack, because it is still standing on it.
10. **The idle thread reaps the producer.** When core 0 is idle, its idle thread takes the terminated thread, checks that no core is still running it, frees its stack, marks it "destroyed," and wakes anyone who was waiting in `join()`. Only then may the joiner free the thread object.

Now the same story with **default affinity**. The consumer is allowed on any core. When the producer makes it ready, the producer's own core may run it immediately if it has higher priority. If not, it stays on the rail, and the next core that goes idle takes it. No IPI is required. That is the normal work-distribution path: idle cores take work from the shared rail.

What the story shows:

- The ready list is the only place a runnable thread waits.
- A blocked thread is not in the ready list at all.

- The kernel lock is held only while the queue and the wait list are being changed, not while the consumer runs.
- The claim flag is what prevents core 0 and core 1 from both running the consumer.
- A pinned thread may need an IPI; an unpinned thread may simply be taken by the next free core.
- A thread that exits is cleaned up by the idle thread, not by itself.

---

1. The short version

µOS++ III does not have a second, separate SMP kernel. It has **one portable kernel that is compiled in two variants**, selected at build time by `OS_USE_SMP_SCHEDULER`:

- the **scalar variant**, the classic single-CPU scheduler with one `current_thread_`, one `lock_state`, and interrupt masking as the only mutual exclusion;
- the **SMP variant**, with one `current_thread_[OS_NCPU]`, an affinity-aware picker over a single global ready list, per-CPU idle threads, and a recursive kernel spinlock with which the cores exclude each other.

The portable kernel lives in `micro-os-plus-iii`. Everything that is a fact about an instruction set, an interrupt controller, a cache, or a board lives in an architecture port and in the board directory. The kernel calls a small, closed set of port functions; the port calls back into the kernel's scheduler. That seam is the subject of this document.

The whole design can be reduced to five invariants. If you remember nothing else, remember these:

1. **One ready list, one kernel lock.** All runnable threads are in `scheduler::ready_threads_list_`. The list is protected by the kernel's critical section (a recursive spinlock, with local interrupts masked).
2. **One current thread per CPU.** `scheduler::current_thread_[cpu]` names the thread that CPU is running. A thread that is current on one CPU is not in the ready list.
3. **A thread is claimable only when its saved context is complete.** The port publishes `thread::context_.port_.stack_ptr` only after the CPU has left that thread's stack. Until then it is `nullptr`, and the picker skips it (except for the CPU's own outgoing thread).
4. **The kernel's data is protected by the kernel lock, not by the application's scheduler lock.** On some ports these are the same object; on AArch64 they are deliberately different. Confusing them is the most common way to introduce an SMP race.
5. **A switch under a lock is either deferred or refused, never performed.** Switching while owning the kernel spinlock would strand the lock's owner and depth on the outgoing thread's stack, and every other core would spin forever.

Everything else in this document is an explanation of those five sentences, the code that implements them, and the places where a board or a port has to provide a machine-specific answer.

How to read this document

- If you are new to SMP, read **SMP in plain English** above, then 2–7.
- If you want the architecture, read sections 2–7.
- If you are bringing up a new board, read sections 8, 13 and 14.
- If you are debugging a hang or a lost wake-up, read sections 5, 7, 9, 11 and 16, then the symptom table in section 20.
- If you want a short answer to a practical question, read sections 18 and 19; section 21 is the printable summary.
- If you want to read the source, section 22 gives the order.
- If you are looking for a file, section 17 is the map.

The source references are to this workspace: `micro-os-plus-iii` (the kernel), plus the sibling ports `micro-os-plus-iii-aarch32`, `-aarch64`, `-cortexm` and `-posix-arch`. The description matches the `smp` branch in this workspace, which includes the `new-critical-sections` work: the kernel's own critical section, per-CPU CPU-time accounting, and the deferred-switch fix on AArch64.

2. What runs where

2.1 The layers

The system is layered so that the portable kernel never contains a machine instruction, and a port never contains a policy decision that belongs to the kernel.

Layer	Lives in	Owns
Kernel	`micro-os-plus-iii/src/rtos`, `include/cmsis-plus/rtos`	Threads, scheduler, clocks, timers, mutexes, condition variables, semaphores, queues, event flags, memory pools, the C API
Port contract	`micro-os-plus-iii/port/smp-common` and each port's `include/cmsis-plus/rtos/port`	Type widths, IRQ-state representation, per-CPU arrays and the spinlock structures
Port implementation	each port's `include/cmsis-plus/rtos/port/os-inlines.h` and `src/rtos/os-core.cpp`	Core id, IRQ mask, atomic operations, kernel lock, context-switch entry, secondary-core start
Board	each port's `test/boards/<board>`	Memory map, MMU, interrupt controller routing, timer rate, core-release mechanism, UART, LED, SD, USB
Application	`test/<board>/<test>` or the user's `main`	Threads, affinities, priorities, synchronisation, the boot call sequence

The important consequence is that **there are two files called `src/rtos/os-core.cpp` in every image**, and both are compiled:

- the kernel's `micro-os-plus-iii/src/rtos/os-core.cpp`, which contains the portable scheduler (`internal_switch_threads`, the affinity picker, the per-CPU idle bookkeeping, the statistics);
- the **port's** copy of `src/rtos/os-core.cpp`, which contains `port::scheduler::start`, `locked`, `reschedule`, `switch_stacks`, `context::create`, and `port_smp_secondary_start`.

They do not collide because they belong to different include/library targets and only one of each is linked; the path is the same because that is where each project's build expects it.

2.2 The compile-time fork

The scalar and SMP schedulers are not selected by a runtime flag. They are different code paths inside the same source files, guarded by `#if defined(OS_USE_SMP_SCHEDULER)`. The scalar path is the upstream behaviour and remains the default:

```
OS_USE_SMP_SCHEDULER undefined        OS_USE_SMP_SCHEDULER defined
---------------------------------     ---------------------------------
current_thread_                       current_thread_[OS_NCPU]
lock_state                            lock_state[OS_NCPU]
ready_threads_list_                   ready_threads_list_  (same list)
one idle thread                       os_idle_thread_core[OS_NCPU]
interrupt mask = mutual exclusion     interrupt mask + kernel spinlock
no IPI                                port_smp_ipi(cpu)
```

`OS_USE_SMP_SCHEDULER` is not normally written by hand. It is defined by the port header when `OS_NCPU > 1` (AArch32, AArch64), or by the build when the board passes more than one CPU (`uos_add_app` adds `OS_USE_SMP_SCHEDULER=1` for `NCPU > 1`). The Cortex-M port uses the same convention: `include-rp2040` and `include-rp2350` expose their SMP declarations only when `OS_USE_SMP_SCHEDULER` is defined, and the build supplies it (together with `OS_NCPU=2`) for the SMP images; the STM32 boards build the same port at `OS_NCPU=1` and never see the SMP code.

This dual-branch arrangement is deliberate. It means the single-core Cortex-M images are still the upstream reference scheduler, and the SMP changes cannot silently alter them.

---

3. The configuration that makes an SMP build

3.1 Board facts, not defaults

Three numbers are facts about the board, and the tree refuses to guess them. The failure mode that motivated this rule is worth recording: when `OS_NCPU` had a default of 4, a board whose build forgot to state its core count silently got a four-CPU scheduler, which is not a wrong number on a single-core board — it is a different operating system. `OS_SMP_IPI_SGI` had a default of 0, which means "local mailbox 0" on a BCM2837 and "GIC SGI 0" on an RK3506: two unrelated things that happened to share a number.

The AArch ports now say, in `include/cmsis-plus/rtos/port/os-c-decls.h`:

```
#ifndef OS_NCPU
#error "OS_NCPU is a board fact: set UOS_BOARD_NCPU in boards/<board>/board.cmake."
#endif
#if (OS_NCPU > 1) && !defined(OS_SMP_IPI_SGI)
#error "OS_SMP_IPI_SGI is a board fact: the interrupt a core uses to ask another to reschedule."
#endif
```

`test_smpl/src/board-contract.cpp` adds the non-preprocessor checks: the RAM window is ordered, `OS_NCPU` is between 1 and 8, the board has a greeting and a banner, and a semihosting build has chosen a trap. Adding a board that has not answered every question does not compile.

3.2 The switch list

Macro	Where it is set	What it means
`OS_NCPU`	`board.cmake` (`UOS_BOARD_NCPU`) or `-DNCPU`	Number of cores the scheduler has
`OS_USE_SMP_SCHEDULER`	Port header when `OS_NCPU > 1`, or `uos_add_app`	Selects the SMP branch of the kernel
`OS_SMP_IPI_SGI` / `OS_SMP_IPI_FIFO` / `OS_SMP_IPI_MHU`	Board CMake / port header	The cross-core reschedule signal
`OS_HAS_PORT_KERNEL_LOCK`	AArch64 `os-c-decls.h`	The port has a kernel lock separate from the application scheduler lock
`OS_HAS_INTERRUPTS_STACK`	Port header	A separate interrupt stack exists
`OS_INTEGER_RTOS_STACK_FILL_MAGIC`	Port header	Stack paint, element-width dependent
`SMP_NO_OWNER`	Port header	Sentinel meaning "the kernel lock is unowned"
`OS_SYSTICK_DIV`	AArch build, optional	Timer interrupts per logical 1 ms tick (2 gives 0.5 ms preemption while the clock stays 1 ms)

The AArch ports also use `PORT_RAM_BASE` / `PORT_RAM_END` from the board's `smp.hpp` to validate a stack pointer before switching to it. Those are board facts too, and the compile-time contract requires them.

3.3 An example

`micro-os-plus-iii-aarch64/test/boards/rpi4b/board.cmake` says:

```
set (UOS_BOARD_NCPU  4)
set (UOS_BOARD_DEFINES
     SOC_BCM2711
     SEMIHOST_TRAP_CHOSEN SEMIHOST_TRAP_HLT
     OS_SMP_IPI_SGI=0
     ...)
```

and `uos_add_app()` turns that into `OS_NCPU=4` and `OS_USE_SMP_SCHEDULER=1` for every application on that board. Nothing in the application source mentions either macro; the test sources include `<smp.hpp>` and use `OS_NCPU` as a constant.

---

4. Shared and per-CPU state

4.1 What the kernel keeps once

These objects are global to the image and are touched by every core:

- `scheduler::ready_threads_list_` — all runnable threads, ordered by priority (head is the highest);
- `scheduler::terminated_threads_list_` — threads waiting for the idle reaper;
- `scheduler::top_threads_list_` — top-level and detached threads;
- the thread objects themselves, their stacks, their wait-list nodes, their mutex links, their `errno`;
- the clock objects (`sysclock`, `hrclock`, `rtclock`) and the timer lists;
- all mutexes, condition variables, semaphores, message queues, event flags and memory pools;
- the C API's static objects.

None of this is per-CPU, and none of it may be touched without the kernel lock. The kernel lock is the only lock that protects them.

4.2 What is per-CPU

Object	Declared in	Purpose
`thread* volatile current_thread_[OS_NCPU]`	kernel `os-core.cpp`	The thread each CPU is running
`volatile state_t lock_state[OS_NCPU]`	port `os-decls.h`	The application's scheduler lock, per core
`volatile state_t kernel_lock_state[OS_NCPU]`	port `os-decls.h`	The kernel's own section, only on ports with `OS_HAS_PORT_KERNEL_LOCK`
`volatile unsigned _port_ctx_pending[OS_NCPU]`	port `os-decls.h`	A switch was requested by an ISR on this CPU
`bool resched_pending[OS_NCPU]`	AArch64 port only	A switch was requested under a lock
`clock::timestamp_t switch_timestamp_[OS_NCPU]`	kernel `os-core.cpp`	CPU-time accounting per CPU
`thread* os_idle_thread_core[OS_NCPU]`	kernel `os-core.cpp`	The idle thread each CPU falls back to
`smp_klock_t _smp_klock`	port `port_sys.cpp`	The recursive kernel spinlock
`smp_tlock_t _smp_tlock`	port `port_sys.cpp`	A reserved non-recursive leaf lock (see 5.3)
`_smp_pub_addr[]`, `_smp_pub_val[]`	port `os-core.cpp`	Deferred-publish slots
`_in_isr[OS_NCPU]`	port `os-core.cpp`	Handler-mode flag, because AArch64/ POSIX have no banked modes
`_port_core_alive[OS_NCPU]`	port `os-core.cpp`	Set on a secondary's first tick
per-core timer, IRQ interface, MMU state, stacks	board / startup	Machine state

The split is the whole point: a core's own scheduling decision is local (`current_thread_[cpu]`, `lock_state[cpu]`, `_port_ctx_pending[cpu]`), while the set of threads it may choose from is global and lock-protected.

4.3 The thread object in SMP

A thread carries both its permanent state and its transient link in the scheduler:

- `state_` — `ready`, `running`, `suspended`, `terminated`, `destroyed`, `initializing`, `destroying`;
- `ready_node_` — the intrusive node in the ready/terminated list;
- `context_.port_.stack_ptr` — the saved frame pointer, or `nullptr` while the thread is running on a CPU or not yet safe to claim;

- `cpu_affinity_` — a bitmask, all ones by default, `1u << 0` for main and for CMSIS-RTOS v1 threads;
- `waiting_node_`, `clock_node_`, `joiner_`, `mutexes_` — the links used by blocking and teardown.

The single most important field is `stack_ptr`. It is both the saved stack pointer and the **claim flag**: `nullptr` means "this context is live in some CPU's registers, do not resume it". That is how two cores cannot run the same thread.

---

5. Locks and critical sections

5.1 Why masking interrupts is not enough

On a single core, disabling interrupts is sufficient mutual exclusion: no ISR and no other thread can run while interrupts are off, so a short read-modify-write is atomic. On several cores, masking interrupts on one core stops only that core. Another core can read and write the same list, the same `ready_node_`, or the same `owner_` field at the same instant.

The SMP kernel therefore adds a **spinlock**. A core that wants to touch shared kernel data:

1. masks its own interrupts (so its own ISR cannot re-enter the section);
2. acquires the kernel lock (so no other core can be inside);
3. touches the data;
4. releases the lock and restores its interrupt state.

Step 2 is the new part. Steps 1 and 4 are the old single-core discipline, kept because local ISRs are still a concurrency source.

5.2 The recursive kernel spinlock

The lock is a three-word structure:

```
typedef struct {
  volatile uint32_t lock;    /* 0 free, 1 held */
  volatile uint32_t owner;   /* CPU index, or SMP_NO_OWNER */
  volatile uint32_t depth;   /* recursion count */
} os_port_smp_klock_t;
```

and one instance, defined by each board's `port_sys.cpp`:

```
smp_klock_t _smp_klock = {0, SMP_NO_OWNER, 0};
```

The raw operations are ISA-specific. AArch32 uses `LDREX`/`STREX`; AArch64 uses `LDAXR`/`STLXR`; Cortex-M uses the RP2040/RP2350 SIO hardware spinlock or `LDREX`/`STREX` on the SSE-200; the POSIX host uses `__atomic_exchange_n`. The shape is always the same:

```
// acquire (AArch64 shown; AArch32 is the same with LDREX/STREX)
while (true) {
  ldaxr  w0, [&_smp_klock.lock];
  if (w0 != 0) continue;
  w0 = 1;
  stlxr  w1, w0, [&_smp_klock.lock];
  if (w1 == 0) break;
}
dmb ish;

// release
dmb ish;
_smp_klock.lock = 0;
dsb ish;
```

On top of the raw lock, the port implements the **recursive** protocol:

```
port_set_lock():                // enter
  mask local IRQs
  cpu = port_cpu_id()
  if (_smp_klock.owner != cpu) {
      _smp_klock_raw_acquire()
      _smp_klock.owner = cpu
  }
  _smp_klock.depth += 1

port_put_lock(saved_irq_state):  // leave
  cpu = port_cpu_id()
  if (_smp_klock.owner == cpu && _smp_klock.depth > 0) {
      if (--_smp_klock.depth == 0) {
          _smp_klock.owner = SMP_NO_OWNER
          _smp_klock_raw_release()
      }
  }
  restore saved IRQ state
```

Recursion is needed because kernel code nests critical sections freely: a mutex operation takes the kernel lock, calls `resume()`, which takes an interrupts critical section, which takes the same lock. The `owner` test makes the second acquisition a depth increment rather than a self-deadlock.

The `owner` field is also why the lock can be held across a context switch only if the depth is migrated. The scheduler avoids that case: `reschedule()` refuses to switch while the current CPU owns the lock (see section 6.4).

5.3 Three critical sections, and what each protects

The kernel exposes three RAII wrappers. They look similar but have different guarantees on different ports.

`interrupts::critical_section`

Masks this core's interrupts and takes the kernel spinlock (through `port_set_lock()`). It is the innermost, shortest, and most universal section. It protects the intrusive lists, the ready list, the wait lists, the clock lists, and any shared field read or written by more than one core.

```
{
  interrupts::critical_section ics;
  // list manipulation, pointer swap, counter update
}
```

On exit it decrements the depth, releases the lock when the depth reaches zero, and restores the saved DAIF/CPSR/PRIMASK/signal mask.

`scheduler::critical_section` (the application's scheduler lock)

Stops **this core** from switching threads. It does not, by itself, protect data shared with another core on AArch64. Its implementation differs by port:

Port	What `scheduler::lock()` does
AArch32	Masks IRQs, takes the kernel spinlock, sets `lock_state[cpu]`
Cortex-M SMP	Masks IRQs (PRIMASK), takes the kernel spinlock, sets `lock_state[cpu]`
POSIX host	Blocks the tick/IPI signals, takes the spinlock, sets `lock_state[cpu]`
AArch64	Sets `lock_state[cpu]`; IRQs are masked only for the flag update, and no spinlock is held

So on AArch32/ Cortex-M/ POSIX the application scheduler lock is **coarse**: it serialises the whole machine. On AArch64 it is **local**: it only prevents this CPU from switching, which is what a scheduler lock is conceptually for.

`scheduler::kernel_critical_section`

This is the section the kernel itself takes around the heap, `new`, `atexit`, thread creation and teardown, and the internals of mutexes, condition variables, semaphores, queues and event flags.

- On a port with `OS_HAS_PORT_KERNEL_LOCK` (AArch64), it maps to `port::scheduler::kernel_lock()`: an interrupts critical section (IRQ mask
- spinlock) plus a per-core `kernel_lock_state[cpu]` flag. The flag makes `scheduler::locked()` return true inside the section, so no thread switch or blocking call can happen. The spinlock protects the data from other cores.
- On every other port it maps to the scheduler lock, because there the scheduler lock already is the kernel lock.

The companion `kernel_uncritical_section` temporarily **releases** the kernel lock while keeping the logical section. It is used only where the kernel must let a switch happen inside an operation: when boosting a mutex owner's priority (which ends in a yield) and when explicitly yielding while waiting for another CPU to release a thread. On return the lock is reacquired. This is not a general-purpose "sleep while holding a lock" facility; it is a carefully audited window, and the code revalidates any pointer it captured before the window (section 11.2 shows the mutex case).

`kernel_lockable` is the same kernel section exposed as a C++ `Lockable`, so the allocators can use it:

```
template <typename T, class Lockable, ...>
class allocator_typed { ... };   // takes the kernel lock per allocation
```

That is what makes `malloc`, `operator new`, and `atexit` safe from several cores. Before this existed, the heap was protected by the scheduler lock, which on AArch64 no longer excludes other cores — a real SMP bug that the per-port kernel lock closes.

The reserved timer leaf lock

The port contract also declares `smp_tlock_t _smp_tlock` and `port_tmr_lock()` / `port_tmr_unlock()`. It is a simple, non-recursive lock word intended for a future timer back-end with a finer-grained queue. In this tree no portable kernel caller takes it; the clock timestamp lists are currently protected by `interrupts::critical_section`. The storage exists on every ARM board so the contract is uniform, but calling it "the timer lock in use" would be inaccurate.

5.4 The rules that keep it correct

These are not stylistic preferences. Each one corresponds to a hang or a corruption that has been observed.

1. **Mask interrupts before reading the CPU id.** A thread can be preempted and resumed on another CPU. If it reads `port_cpu_id()`, then takes a preemption, then writes `lock_state[cpu]`, it marks the CPU it left as locked and leaves its new CPU unlocked. The matching unlock then finds

"unlocked" and never releases the kernel lock. The Cortex-M and POSIX ports call this out explicitly in their `locked()` implementations.

2. **Clear `owner` / `depth` before clearing the lock word.** Every release site does this in one order: set `depth = 0`, set `owner = SMP_NO_OWNER`, execute `dmb`, store `0` to `lock`, execute `dsb`. Reversing the first two stores opens a two-instruction window in which another core wins the lock, installs its own owner/depth, and then has them wiped by the first core's stores. Its `exit()` is guarded by `owner == cpu && depth > 0`, so it never releases the word again; every core spins forever. This exact bug was the root cause of an `smp_test4` hardware deadlock on AArch32 and an `smp_test3` stall on AArch64.
3. **Never switch while owning the kernel lock.** The owner and depth would be saved with the outgoing stack, and the incoming thread would inherit a lock it does not own. `reschedule()` checks `_smp_klock.owner == cpu` and either sets a pending flag or returns; the actual switch waits until the section is left.
4. **Never call a blocking kernel function inside an interrupt or kernel section.** `lock()`, `wait()`, `sleep_for()`, `join()` and `kill()` assert that the scheduler is not locked or that the call is not in handler mode.
5. **A critical section is short.** Other cores spin with their own interrupts masked while they wait. A long section (a UART write, an SD transfer, a debug print) turns into a machine-wide stall. That is why `kernel_uncritical_section` exists for the few legitimate long windows, and why the tests move console output outside the kernel.
6. **No spinlock is a priority-inheritance lock.** The kernel lock relies on the holder having interrupts masked, so the holder cannot be preempted on its own core and finishes the section. It is not a general-purpose lock for application data; use a µOS++ mutex for that.
7. **The application scheduler lock is not a substitute for the kernel lock.** On AArch64 it does not exclude other cores at all. Application data shared between cores needs a mutex, an atomic, or an explicit kernel section — not `scheduler::critical_section` alone.

5.5 Memory ordering

The ports use the architecture's acquire/release primitives and explicit barriers. The pattern is:

Port	Acquire	Release	Notes
AArch32	`ldrex` / `strex` + `dmb`	`dmb` before store, `dsb` after	ARMv7 `dmb` is full system
AArch64	`ldaxr` / `stlxr` + `dmb ish`	`dmb ish` before, `dsb ish` after	Inner-shareable domain
Cortex-M (RP2040/RP2350)	SIO hardware spinlock + `dmb`	`dmb` before, `dsb` after	The SIO lock is the hardware arbiter
Cortex-M (SSE-200)	`ldrex` / `strex` + `dmb`	`dmb` / `dsb`	Same shape as AArch32
POSIX	`__atomic_exchange_n(..., ACQUIRE)`	`__atomic_store_n(..., RELEASE)`	Host compiler's atomics

Two more rules matter:

- **The deferred-publish store is a release.** The CPU stages the outgoing frame pointer, moves `SP` to the incoming stack, then stores the pointer and executes a `dmb` / `dmb ish`. A core that sees the non-null pointer is guaranteed to see the saved frame it points at.
- **The claim is a clear.** The incoming thread's `stack_ptr` is set to `nullptr` under the kernel lock before the CPU returns into it. No other core can pick it once it is running.

6. Scheduling

6.1 The ready list and the picker

The ready list is a single intrusive doubly-linked list, sorted by priority. `link()` inserts a thread at the end of its priority band, so equal-priority threads are FIFO and the head is always the highest-priority runnable thread. The list is global; there are no per-CPU run queues.

A context switch is the same operation on every core: the outgoing thread is re-linked if it is still runnable, then the picker scans from the head for the first thread that is allowed on this CPU and whose context is published.

```
// os-core.cpp, internal_switch_threads(), SMP branch (condensed)
unsigned cpu = port_cpu_id();
thread* old = current_thread_[cpu];

bool old_is_idle = (old == os_idle_thread_core[0..N-1]);
if (!old_is_idle)
    old->internal_relink_running_();   // re-adds only if state_ == running
else
    old->state_ = state::ready;        // idle is never in the ready list

thread* next = nullptr;
for (node = ready_threads_list_.head(); node != sentinel; node = node->next()) {
    thread* th = node->thread_;
    if (th == nullptr) continue;
    if (!is_thread_allowed_on_cpu(th, cpu)) continue;
    if (th == old || (th->state_ != state::running
                      && th->context_.port_.stack_ptr != nullptr)) {
        next = th;
        node->unlink();
        next->state_ = state::running;
        break;
    }
}

current_thread_[cpu] = next ? next : os_idle_thread_core[cpu];
if (!next) os_idle_thread_core[cpu]->state_ = state::running;
```

The important details:

- `internal_relink_running_()` only re-links a thread whose state is still `running`. A thread that blocked, exited, or was marked `suspended` before the switch is **not** re-added. That is how blocking works without a separate "remove from ready list" step.
- The `stack_ptr != nullptr` test is the claim gate. A running thread has `nullptr` and is skipped, even if a stale ready node existed.
- `th == old` is the exception: the CPU is allowed to re-pick the thread it just left, because that context is still live on this CPU and was never published.
- If nothing is eligible, the CPU runs its idle thread. The picker does not spin looking for work; it goes idle and waits for a tick or an IPI.
- The thread that runs is removed from the ready list and marked `running`. When it later switches out, it is re-linked at its priority position.

This is a **global queue with affinity filtering**, not a work-stealing scheduler with per-CPU queues. Its virtues are simplicity and exact priority ordering; its cost is a single hot lock around every scheduling decision.

6.2 Affinity and idle threads

`thread::cpu_affinity()` is a 32-bit mask. The default is `0xFFFFFFFF`, so a thread may run on any CPU. `os_main` is pinned to CPU 0, and the CMSIS-RTOS v1 compatibility layer pins every thread it creates to CPU 0, because the v1 API assumes a single-core priority model.

The affinity rule is not a plain mask test. It is, in order:

```
static bool is_thread_allowed_on_cpu(thread* th, unsigned cpu) {
    for (c = 0; c < OS_NCPU; ++c)
        if (th == os_idle_thread_core[c])
            return cpu == c;                        // registered idle: pinned

    // boot-window fallback: an idle thread is linked by its constructor
    // before the application registers it in os_idle_thread_core[]
    if (name starts with "idle") {
        if (name is "idle" or "idle0") return cpu == 0;
        if (name is "idle1".."idle9")  return cpu == name[4] - '0';
    }

    return (th->cpu_affinity() & (1u << cpu)) != 0;
}
```

Two things are being protected here. First, a per-CPU idle thread must never run on another CPU: its stack is one stack, and two CPUs using it would corrupt each other. Second, there is a boot window between the moment the idle thread object is constructed (and linked into the ready list) and the moment the application stores it in `os_idle_thread_core[]`. The identity test does not cover that window, so the name rule does. The comment in `os-core.cpp` is precise about why the last core's idle thread would otherwise inherit `0xFFFFFFFF` and be pickable by every core.

The idle threads themselves are application-supplied for the secondary cores. The kernel creates `os_idle_thread` for CPU 0 during `os_startup_create_thread_idle()`. On the AArch ports the board's `test-smp-boot.cpp` creates `idle1`…`idleN` and registers them; the Cortex-M tests create their `idle1` directly in the test source. Either way they are registered before the core is released. A secondary that is released without a registered idle thread has no fallback: the picker returns `nullptr`, and the port halts with `no ready thread and no idle thread on Core N`.

6.3 Preemption and time slicing

The default tick is 1 ms (`OS_INTEGER_SYSTICK_FREQUENCY_HZ`), driven by each core's private timer. The timer ISR does two things:

1. If this is CPU 0, call `os_systick_handler()` to advance the system clocks and expire timeouts. On other CPUs, only reschedule.

2. Set `_port_ctx_pending[cpu] = 1` and, on the way out of the handler, take a context switch if the core is not locked.

So every core gets a preemption point every tick, and equal-priority threads on the same core are round-robin: the outgoing thread is re-linked at the end of its band and the next one at the head runs.

There are three preemption paths:

Path	What happens	Latency
Current core, thread-mode `reschedule()`	Calls the switch immediately (or at ISR exit)	Immediate
Current core, wake-up of a higher-priority thread	`resume()` calls `reschedule()`; the new thread runs before the waker continues	Immediate
Another core, thread made ready	No synchronous remote preemption; the target's tick/IPI picks it up	≤ 1 tick, or an IPI if the kernel sends one

The kernel sends an IPI in two places:

- `thread::resume()` — if the waking CPU is **not** in the woken thread's affinity mask, it sends an IPI to the first allowed CPU, so the thread does not wait a full tick for a core it can actually use;
- `thread::kill()` — when the thread to be destroyed is current on another CPU, it marks the thread `suspended` under the kernel lock and sends an IPI to that CPU so its switch will leave the thread off the ready list.

A woken thread whose affinity includes the waker runs on the waker's core immediately if it has higher priority. A woken thread with default affinity and no higher priority stays in the ready list; another core that goes idle will pick it up. That is the whole of the load-balancing mechanism: **idle cores steal from the global ready list; running cores are never migrated.**

6.4 A switch requested under a lock

A thread may make another thread ready while it holds a lock: a mutex unlock resumes a waiter, a queue send resumes a receiver, a priority change makes a thread eligible. Switching at that instant would be wrong for two reasons: the lock's owner/depth would be stranded, and the caller's data structures might be half-updated.

The ports handle this in two ways:

- **AArch64** keeps `resched_pending[cpu]`. `reschedule()` under the scheduler lock or the kernel section sets it and returns. When the last lock is released (`scheduler::locked(unlocked)` or `kernel_locked(unlocked)`), the port takes the pending switch if the CPU is in thread mode. This gives an immediate switch at the end of the section, not at the next tick.
- **AArch32, Cortex-M and POSIX** rely on the tick. POSIX additionally sets `_port_ctx_pending[cpu]` if it finds the kernel spinlock still held, and leaves the switch to the ISR exit or the next tick. On AArch32

and Cortex-M the scheduler lock and the kernel lock are the same object, so `reschedule()` returns at the first `locked()` test; the request is deferred to the next tick there.

The AArch64 `resched_pending` flag is deliberately separate from `_port_ctx_pending`. The tick's time-slice request must **not** be honoured at the end of every kernel section: doing so broke equal-priority round-robin in the middle of a heap allocation, because every allocation boundary became a preemption point. Only a switch that a kernel operation explicitly asked for is honoured when the kernel section ends.

6.5 What happens when no thread is ready

The CPU runs its idle thread. The kernel's `os_idle` loop:

1. calls `os_rtos_idle_actions()`, which reaps terminated threads (section 12) and otherwise does nothing;
2. calls `port::scheduler::wait_for_interrupt()` (`dsb; wfi` on ARM, `sigsuspend` on POSIX);
3. calls `this_thread::yield()` to re-run the picker.

If another core makes a thread ready and the idle core is in `wfi`, the next tick (or an IPI) wakes it. On ARM the timer keeps running, so the idle core does not sleep forever.

6.6 Statistics

When `OS_INCLUDE_RTOS_STATISTICS_THREAD_CPU_CYCLES` is enabled, each CPU has its own `switch_timestamp_[OS_NCPU]`. A switch charges the outgoing thread the time since **that CPU's** previous switch, not since the last switch of any CPU. Before the per-CPU array existed, with four busy cores each thread was charged about a quarter of its real CPU time, and all threads together added up to one times the elapsed time instead of four. The fix is a good example of an SMP bug that is invisible on one core.

`context_switches_` is a single global counter; all switches hold the kernel lock, so the increment is serialised. Per-thread counters are updated under the same lock.

---

7. Context switching and the deferred publish

7.1 The hazard

On AArch32 and AArch64 the IRQ/exception handler runs on the **interrupted thread's own stack**. The sequence of a preemptive switch is:

1. the exception entry saves the interrupted frame on the current stack;
2. C++ code selects the next thread;

3. the restore path loads the incoming stack pointer;
4. the restore path pops the incoming frame and returns.

If the outgoing thread's saved stack pointer were published in step 2, another core could pick that thread and start running it **while this core is still executing on its stack** — saving registers, calling C++ destructors, popping the frame. The two cores would write the same stack. The corruption is intermittent and looks like a wild branch or a corrupted return address.

The kernel's answer is to make the saved context invisible until the CPU has physically left the stack. This is the **deferred publish** protocol. It is the single most delicate part of the SMP port.

7.2 The protocol

The picker's claim flag is `thread::context_.port_.stack_ptr`. Its states are:

Value	Meaning
`nullptr`	The thread is running on some CPU, or its context is being saved/restored. Not claimable.
non-null	A complete frame exists at that address; any allowed CPU may claim it.

The switch code in `port::scheduler::switch_stacks()` does:

```
old = current_thread_[cpu]
take the kernel lock (owner = cpu, depth = 1)
internal_switch_threads()             // old re-linked, next chosen
new = current_thread_[cpu]

if (new != old):
    check new->stack_ptr lies in new's stack and has a valid mode word
    _smp_pub_addr[cpu] = &old->context_.port_.stack_ptr
    _smp_pub_val[cpu]  = sp                    // the outgoing frame
    out_sp = new->stack_ptr
    new->stack_ptr = nullptr                    // CLAIM
else:
    out_sp = sp
    _smp_pub_addr[cpu] = 0

release the kernel lock (owner/depth, then lock word)
return out_sp
```

The assembly restore path then:

```
mov sp, out_sp                // SP is now off the outgoing thread's stack
SMP_PUBLISH                   // store _smp_pub_val[cpu] into *_smp_pub_addr[cpu]
... restore the incoming frame, return
```

Only after `sp` has moved does the outgoing frame become visible. The store is one aligned pointer plus a `dmb`, so another core sees either `nullptr` (not claimable) or the complete frame (claimable), never a half-written pointer.

The double handshake is worth spelling out as a timeline:

```
CPU A (leaving thread T)                  CPU B (looking for work)
------------------------------            ------------------------
T is current; T.stack_ptr == null
choose U; U.stack_ptr == null (claim U)
stage T's frame in pub[A]
release the kernel lock
mov sp, U's stack                          takes the kernel lock
SMP_PUBLISH: T.stack_ptr = frame           scans the ready list
dmb                                        sees T.stack_ptr != null
run U                                      claims T; T.stack_ptr = null
                                           runs T
```

There is a second, subtler case: the **same** thread is re-picked. Then the CPU never left it, so the frame is still live in its registers; no publish is needed, and `_smp_pub_addr[cpu]` is explicitly zeroed. If a stale slot from an earlier switch were left in place, the next restore would publish an old frame into a thread that may have moved on.

The POSIX port has the same hazard in a different form. `swapcontext()` returns only in the **incoming** context, so the CPU that left cannot do the publish itself. It stages the slot and the first context to run on that CPU does `host_cpu::publish_pending()` after the switch (or in the trampoline for a never-run context). Same rule, different instruction.

The Cortex-M port does **not** need deferred publish. Its PendSV handler runs on the MSP (the interrupt stack), and the outgoing thread's PSP frame is quiescent as soon as `save_on_stack()` returns. `switch_stacks()` publishes it immediately. That is a genuine architectural simplification, not an oversight.

7.3 The saved frame, per port

Port	Frame	What is saved	Mode check
AArch32	`ctx_t` in `port_ctx.hpp`	r0–r12, lr, cpsr, pc, fpscr, pad, d0–d31 (full NEON)	CPSR mode == SVC (0x13)
AArch64	800-byte `ctx64`	x0–x30, spsr, elr, fpsr, fpcr, q0–q31	SPSR mode == EL1h
Cortex-M	hardware frame + software `frame_t`	r0–r3, r12, lr, pc, xPSR, r4–r11, EXC_RETURN	EXC_RETURN / PSP
POSIX	`ucontext_t`	Host register set, signal mask, FPU state	Host `swapcontext`

The AArch32 and AArch64 ports also save the **full** floating-point/NEON register file, not just the callee-saved half. A preemptive switch can land in the middle of NEON code, and the thread may resume on another core; saving only d0–d15 or q0–q15 silently corrupts the upper half.

Both AArch ports run a **tripwire** on the incoming frame before switching to it:

- the pointer must lie inside the incoming thread's own stack;
- the saved mode word must be the thread mode (SVC on AArch32, EL1h on AArch64).

A violation prints the old/new thread names and the claimed pointer, then halts. This turns a publish-protocol violation into a loud failure at the source instead of a wild jump later. The bounds come from `PORT_RAM_BASE` / `PORT_RAM_END` in the board's `smp.hpp`, not from a hard-coded address.

The AArch32 `irq_handler` and `port_ctx_switchHandler` also execute `clrex` before returning to a thread. An `LDREX` reservation belongs to a core, not a thread; a thread preempted between its `LDREX` and `STREX` could otherwise have its `STREX` succeed against a reservation taken by whoever ran on that core in between. The ARM ARM requires a `CLREX` at a context switch.

7.4 The first switch

Before the first real thread exists, `port::scheduler::start()` installs a fake `os_thread_t` as `current_thread_[cpu]` (one per core, static storage, zeroed). It has `state_ == 0` (not `running`), so `internal_relink_running_()` ignores it; it is only a place for the first switch to save the boot context. The first `reschedule()` then selects the first ready thread (usually `main` or the idle thread) and never returns to the fake context.

The same pattern is repeated by `port_smp_secondary_start()` on each secondary core: a static `fake_thread_secondary[cpu]` becomes `current_thread_[cpu]`, then `reschedule()` selects that core's idle or a real thread. The fake thread is never restored.

---

8. Booting the other cores

8.1 The sequence

On AArch32/AArch64, core 0 boots from the reset vector; the other cores are held by the board until core 0 releases them. The application controls the release, because only the application knows which threads and which peripherals the secondary cores will use.

The canonical AArch sequence, used by every AArch SMP test in this tree, is:

```
core 0 (reset -> _start -> main):
    initialise the board (UART, MMU, heap, exceptions)
    scheduler::initialize()
    create the main thread, register it as os_main_thread
    os_startup_create_thread_idle()          // kernel idle thread, CPU 0
    scheduler::start()                        // first switch into main

core 0 (os_main, after the scheduler is running):
    unmask interrupts
    smp_install_boot_threads()                // create idle1..idleN-1,
                                              // register in os_idle_thread_core[]
    smp::start_secondary_cores()              // release cores 1..N-1,
                                              // one at a time, waiting for
                                              // _port_core_alive[c] between
    ... create workers, pin them, run the test ...

secondary core (released):
    _start -> secondary_park -> coreN_entry -> boot_coreN
    port_smp_secondary_start():
        mask interrupts
        port_sys_init()                       // this core's timer + IPI
        current_thread_[cpu] = fake_thread_sec
        reschedule()                          // into idleN or a real thread
        unmask interrupts
        for (;;) wfi
```

The order is not cosmetic. Releasing a core before its idle thread is registered means the picker has nothing to fall back to and the port halts. Releasing all cores at once used to create a race on the first switch's idle claim; the release loop now waits for `_port_core_alive[core]`, which is set on that core's first 1 ms tick, before releasing the next.

The tests unmask interrupts with `(void)os::rtos::interrupts::uncritical_section::enter()`. That calls the static `enter()` and deliberately does **not** construct the RAII object, so the saved state is not

restored at the end of the statement; the core stays unmasked. Constructing the object instead would re-mask at the closing brace.

8.2 How each board releases a core

Board	Mechanism
Raspberry Pi 3 / Zero 2 W (BCM2837)	Spin table or GPU armstub mailbox: write the entry (`_start` or `coreN_entry`) to the core's mailbox, clean the line to the point of coherency, `sev` . The parked core polls with MMU/ caches off, so the cache clean is mandatory on real hardware.
Raspberry Pi 4 B (BCM2711)	AArch64 spin table at `0xD8 + 8*core` : write `coreN_entry` to `__smp_spin[core]` , write `&_start` to the release word, clean the line, `dsb` , `sev` .
Luckfox Lyra (RK3506)	SRAM mailbox word `0x00200000` + magic, then clear the core's reset bits in `CRU_SOFTRST_CON00` , `dsb` , `isb` , `sev` .
Raspberry Pi Pico (RP2040)	Bootrom inter-core FIFO handshake: `multicore::launch_core1(port_smp_secondary_start, port_core1_stack_top())` . The bootrom loads the given MSP. The FIFO IRQ must stay disabled during the handshake, or the handler would drain the bootrom's words.
Raspberry Pi Pico 2 (RP2350)	Same bootrom FIFO handshake as the RP2040, with the RP2350's shared SIO FIFO IRQ 25 instead of the RP2040's per-core 15/16.
QEMU `2xcortex-m33` (SSE-200)	The platform's CPU1 release, with the MHU as the IPI.
POSIX native	`pthread_create()` one host thread per CPU; each sets its CPU index, masks the IRQ signals, builds a fake context, starts its tick, and reschedules.

The common pattern is: **park with caches off and poll a word that core 0 writes**, then jump to an entry that sets up banked stacks and the MMU (or MSP) before entering C++. The cache clean is not paranoia: a parked core reads through caches that are off, while core 0 writes through a write-back cache. Without the clean, the write can stay in core 0's L1 and the secondary never sees it.

8.3 The join handshake

There are two different "joins" in the tree, and they are easy to confuse:

- `_port_core_alive[core]` is a port-level flag, set in each core's timer ISR on its first tick. It means "this core has completed bring-up and is scheduling with interrupts on". The board's release loop waits on it before releasing the next core.

- `g_core_stage[core]` is an application/test-level stage counter. The board's `boot_coreN()` sets it to 3 when it enters `port_smp_secondary_start`; the test's `test_wait_secondaries()` and `test_join_summary()` use it for reporting. It is not used by the kernel.

Keeping these separate matters: a board can be correct about `_port_core_alive` while a test's `g_core_stage` is not yet written, and the test should report a join timeout rather than assume the core is dead.

8.4 The secondary's own state

`port_smp_secondary_start()` is board-independent and lives in the port's `src/rtos/os-core.cpp`. It:

1. masks interrupts;
2. calls `port_sys_init()`: on AArch32/AArch64 this routes the private timer and the IPI to this core and starts the 1 ms tick; on the Cortex-M ports it sets PendSV/SysTick priorities and enables the FIFO/MHU IRQ; on POSIX it arms the per-thread timer and installs the signal handlers;
3. installs the fake current thread;
4. calls `reschedule()` to enter the scheduler;
5. unmasks and parks in `wfi`/`pause` if the scheduler ever returns.

The secondary does not run `main`, does not re-run global constructors, and does not re-initialise shared devices. Anything shared must already have been initialised by core 0 under its own boot, or be per-core.

---

9. Inter-processor interrupts

An IPI is how one core asks another to reschedule now instead of at its next tick. The port supplies one function:

```
void port_smp_ipi(unsigned cpu);
```

Its implementations are:

Port	Signal
AArch32 BCM2837	Mailbox 0 of the target core (`bcm_irq::send_ipi`)
AArch32 RK3506	GIC-400 SGI `OS_SMP_IPI_SGI` (`gic::send_sgi`)
AArch64 BCM2837	Mailbox 0 of the target core
AArch64 BCM2711	GIC-400 SGI `OS_SMP_IPI_SGI`
Cortex-M RP2040/RP2350	SIO inter-core FIFO; the receiver's FIFO IRQ drains it and pends PendSV
Cortex-M SSE-200	Message Handling Unit (MHU) doorbell
POSIX	`pthread_kill(target, SIGRTMIN+1)`

The receive side is a normal ISR that does two things: acknowledge the source, and set `_port_ctx_pending[cpu] = 1`. When the handler exits, the same epilogue as the timer path calls `port_switch_stacks()` if the core is not locked and does not own the kernel spinlock. The IPI therefore costs one interrupt entry plus one scheduling decision, not a separate mechanism.

The kernel sends an IPI in exactly two situations (section 6.3): a wake-up whose target affinity excludes the waker, and `kill()` of a thread current on another CPU. There is no periodic "reschedule everyone" broadcast; a busy core is preempted by its own tick.

---

10. Time on multiple cores

Time is deliberately centralised on CPU 0.

- Each core has its own private timer (ARM generic timer PPI on AArch32/ AArch64, SysTick on Cortex-M, a per-thread POSIX timer on the host). Each timer fires at the tick rate and drives that core's preemption.
- **Only CPU 0 calls `os_systick_handler()`.** That function increments `sysclock` and `hrclock`'s tick-driven count, walks the clock timestamp lists, and fires timers. If every core incremented the clock, time would run N times too fast.
- The other cores still take their timer interrupt and still reschedule. Their tick is a preemption point, not a clock update.

`sysclock` is therefore a single global 1 ms counter, advanced under the kernel lock. `clock::now()` and `clock::steady_now()` read it under an interrupts critical section.

`hrclock` is the high-resolution clock. On AArch32/AArch64 it reads the system counter (`CNTPCT`) and subtracts `timer_arm::hr_base()`, the counter value when CPU 0 started its tick. On Cortex-M there is no free-running system counter, so the port reports `has_hardware_counter() == false` and the kernel synthesises the value from `steady_count_ + cycles_since_tick()`.

The AArch timer re-arm is **absolute**, not "one period from now": it adds one period to the previous compare value. That avoids accumulating interrupt latency into every tick. If a core was halted (by a

debugger, or by a slow semihosting call) and many periods have passed, the re-arm catches up rather than replaying thousands of interrupts. On CPU 0 the skipped periods also move `hr_base` forward, so `hrclock.now()` does not stay ahead of the tick-driven clock it is compared with. This matters on boards where a semihosting console trap stops the cores: the clock would otherwise drift by seconds.

Timer callbacks (the `timer` class) run from CPU 0's tick, in interrupt context. They are not threads. The callback is invoked outside the clock list's lock, but its state check and re-arm take the same `interrupts::critical_section` that `timer::stop()` and the destructor take. That is why `internal_callback()` re-checks the state and re-arms under the lock rather than touching the object after it.

---

11. Kernel objects and inter-thread communication

11.1 Waiting and waking

All blocking objects use the same two lists:

- a **waiting list** (`internal::waiting_threads_list`) of threads blocked on the object;
- optionally a **clock list** (`internal::clock_timestamps_list`) for timed waits.

A thread blocks by putting a `waiting_thread_node` (or `timeout_thread_node`) on its own stack, linking it under an interrupts critical section, setting its state to `suspended`, and calling `reschedule()`. The node is unlinked on the way out if the wake-up did not already do it. Because the node lives on the waiter's stack, the waiter must not return until the node is unlinked; the unlink paths are written with that in mind.

A waker unlinks the head waiter under the lock, then calls `thread::resume()` outside the lock. `resume()` re-links the thread into the ready list under an interrupts critical section and calls `reschedule()`. On SMP the woken thread may run immediately on this core, or on another core after the IPI/tick.

`resume_all()` is the broadcast path. Its implementation was changed after a deadlock:

```
// old: while (resume_one()) ;   -- can loop forever
waiting_threads_list now;
{
  interrupts::critical_section ics;
  move the current waiters to `now`;   // O(1) splice
}
while (now.resume_one()) ;
```

The reason is visible only on SMP (and on a fast single core): a woken thread whose condition is still false re-links itself into the **same** list. With `while (resume_one())`, the loop wakes the same thread again and

again, and if the waiter runs above the caller it can starve the caller forever. Moving the current waiters to a local list first bounds the wake-up to the threads that were waiting when the call began. This fixed `event_flags::raise()` and the reset paths of semaphores, mutexes, memory pools and message queues.

11.2 Mutexes and priority inheritance

A mutex owner is recorded in `owner_`. `internal_try_lock_()` may boost the owner's inherited priority when a higher-priority thread blocks on it. The boost is a cross-core operation: the owner may be running on another CPU.

The SMP hazard is the `kernel_uncritical_section` window around `owner->priority_inherited()`. Releasing the kernel lock lets the owner finish its `unlock()` on another CPU and set `owner_ = nullptr`. If the boosting code then dereferences `owner_->...`, it dereferences null. The fix is to capture the owner pointer while the lock is held, pass the captured pointer into the window, and **revalidate** `owner_ == owner` before using it. If the owner changed, the boost is abandoned and the lock attempt restarts.

This is a template for any SMP code that opens a `kernel_uncritical_section`: capture what you need before the window; after the window, re-read the state and handle the fact that another core may have changed it.

A mutex unlock resumes exactly one waiter, then clears the owner and count. The `list_.resume_one()` call happens inside the same kernel critical section as the owner update, so the mutex state and the waiter's readiness are consistent.

11.3 Condition variables, semaphores, queues, flags

The pattern is the same for all of them:

- the object's state and its wait list are protected by the kernel/IRQ critical section;
- a waiter links a stack node, suspends, and reschedules;
- a signaller updates the state, unlinks a waiter (or all current waiters for a broadcast), and resumes them;
- a timed wait also links a clock node, and the timer that expires it unlinks it under the same lock.

Specific notes:

- **Condition variables** use `resume_one()` for `signal()` and `resume_all()` for `broadcast()`.
- **Counting semaphores** keep a count and a wait list; `post()` increments the count or resumes a waiter.
- **Message queues** copy the message into a slot under the lock, then resume a receiver. The four loops (`send`, `timed_send`, `receive`, `timed_receive`) reschedule after **every** attempt, whether it succeeded on the first try or after waiting. Without that, a higher-priority thread resumed by the successful operation stayed ready while the caller went on, until the caller's next kernel call or the tick.

- **Event flags** use `resume_all()` on `raise()`; the move-the-list fix is what keeps `raise()` from looping when a woken thread's mask is not yet satisfied.
- **Memory pools** use the same wait-list and kernel-lock discipline for fixed-size blocks.

11.4 Memory and libc

The heap, `malloc`, `operator new/new[]`, `atexit`, and the memory resources all take `scheduler::kernel_lockable` around their shared state. On AArch64 this is the dedicated kernel section; on the other ports it is the scheduler lock. That is what makes allocation safe when several cores allocate at once.

The kernel's default memory resources and the C++ `pmr` resources are process-wide, not per-CPU. Do not assume a per-core heap.

11.5 `errno` and `this_thread`

`errno` is stored in the thread object, and `__errno()` returns the current thread's. That is correct because `this_thread::_thread()` reads the CPU id and `current_thread_[cpu]` inside a single interrupts critical section:

```
interrupts::critical_section ics;
th = scheduler::current_thread_[port_cpu_id()];
```

Without the mask, a preemption between the two reads could move the caller to another core and return the thread now running on the core it left — wrong mutex owner, wrong errno, wrong wait list.

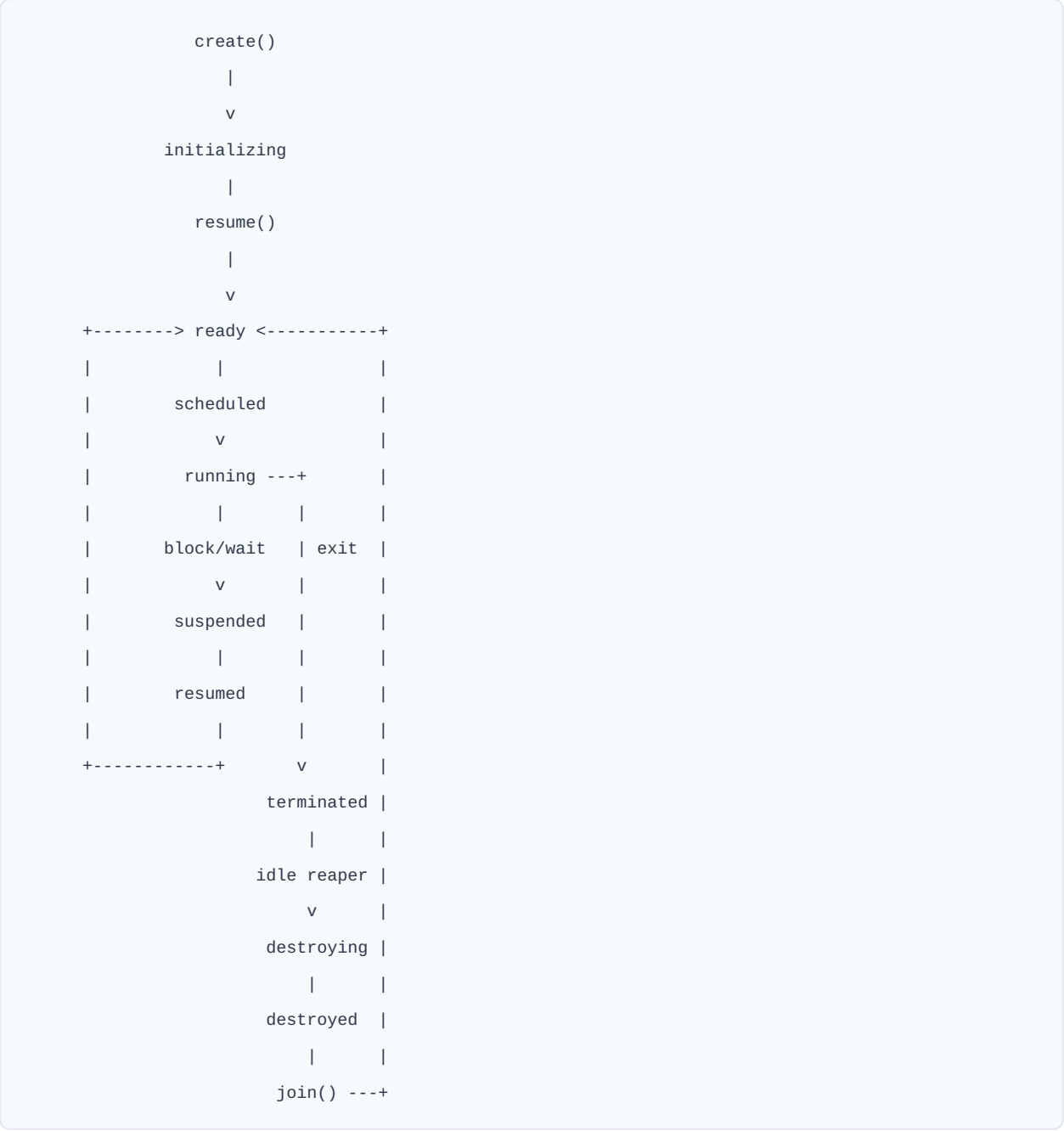
The POSIX port has an extra twist. There, a host thread **is** a CPU, and a µOS++ thread can migrate between host threads at a switch. Native TLS (`thread_local`, `errno`, the address of `_this_cpu`) belongs to the host thread, not to the µOS++ thread. The port therefore:

- reads `_this_cpu` through a non-inlined `port_cpu_id()` so the compiler cannot cache its address across a switch;
- saves and restores `errno` around the signal-driven switch, out of line;
- documents that no port or application state may live in native TLS across a switch point.

That is a host-port-specific rule, but it is the clearest illustration of what "a thread is not a CPU" means in an SMP kernel.

---

12. Thread lifecycle on SMP



The SMP-specific parts are:

Create. `thread::create` builds the port context, links the ready node, and calls `resume()`. If the scheduler has not started, it also installs the new thread as `current_thread_[port_cpu_id()]`; `scheduler::start()` replaces that with the boot fake thread, so the temporary assignment is harmless. The `th_cpu_affinity` attribute is copied into `cpu_affinity_`.

Resume. Re-links the ready node only if the state is `suspended` or `initializing`. A running thread has `ready_node_.next() == nullptr` on SMP and must not be re-enqueued while it is executing. Then it calls `reschedule()`, and possibly the affinity IPI described in section 6.3.

Yield/sleep. `this_thread::yield()` calls `reschedule()`; `sleep_for` links a timeout node and suspends. The thread is not in the ready list while suspended.

Exit. `internal_exit_()` unlinks the thread from the ready list, clears its function and arguments, checks that it owns no mutexes and has no children, then links its ready node into `terminated_threads_list_` and reschedules. It does **not** free its own stack: it is still running on it.

Reap. The kernel's idle thread (`os_idle`) calls `os_rtos_idle_actions()`, which pops terminated threads and calls `internal_destroy_()`. The reaper must not destroy a thread that is still current on another CPU or whose context has not been saved; it checks `stack_ptr == nullptr` and scans `current_thread_[0..N-1]` before claiming the thread. If the thread is still live, it leaves it linked for the next idle pass. This is the same claim/publish rule as the ready list, applied to teardown.

Join. `join()` registers the joiner, suspends it, and waits until the target reaches `destroyed`. On SMP it then waits, with `yield()`, until the target is no longer `current_thread_[c]` on any CPU. Only then may the caller free the thread object and its stack. Without that second wait, the joiner could free the stack of a thread that is still finishing its final switch.

Kill. `kill()` of a thread current on another CPU:

1. marks the thread `suspended` under the kernel lock (so the target's next switch does not re-link it);
2. sends an IPI to the CPU running it;
3. waits, yielding with the kernel lock released, until the target is off every CPU and its context is saved;
4. claims it for destruction.

Destroying a thread that is still running on another core frees the stack that core is executing on; that is the bug this protocol prevents.

Destroy. `internal_destroy_()` frees the allocated stack, marks any mutexes the thread still owns as owner-dead and unlocks them, sets `state::destroyed` under the lock, and wakes the joiner if there is one. From `destroyed` on, the joiner may free the object, so the wake-up must happen under the same lock that the joiner reads.

Idle. CPU 0's idle thread is the kernel's `os_idle`; secondary cores use the application's idle function. If the application's secondary idle does not call `os_rtos_idle_actions()`, only CPU 0 reaps terminated threads. That is the default in the tests, and it is a deliberate simplification: one reaper is enough, and it avoids two cores racing to destroy the same thread.

---

13. The ports compared

	AArch32	AArch64	Cortex-M (RP2040/ RP2350/SSE-200)	POSIX host
ISA	ARMv7-A	ARMv8-A (EL1)	ARMv6-M / ARMv8-M	x86-64 / AArch64 host
Example boards	Pi 3B, Zero 2 W, Luckfox Lyra	Pi 3B, Zero 2 W, Pi 4B	Pico, Pico 2, QEMU M33	`native`
Core id	`MPIDR` via CP15	`MPIDR_EL1`	SIO `CPUID` / SSE-200 CPUID	`thread_local _this_cpu`
IRQ mask	`cpsid i` / `cpsie i`	`msr daifset/ daifclr, #2`	`cpsid i` / PRIMASK	block tick/IPI signals
Kernel lock	`LDREX/STREX` word	`LDAXR/STLXR` word	SIO hardware spinlock / `LDREX/ STREX`	`__atomic_exchange_ n`
Application scheduler lock	= kernel lock (IRQ mask + spinlock)	per-core flag, IRQs on, no spinlock	= kernel lock (IRQ mask + spinlock)	= kernel lock (signal mask + spinlock)
Dedicated kernel lock	no (`OS_HAS_PORT_KERNEL_LOCK` absent)	yes	no	no
IPI	mailbox 0 (BCM) / SGI (GIC)	mailbox 0 (BCM) / SGI (GIC)	SIO FIFO / MHU	`pthread_kill`
Switch	IRQ or cooperative asm	IRQ or cooperative asm	PendSV on MSP	`swapcontext`
Deferred publish	yes, asm after `mov sp`	yes, asm after `mov sp`	not needed (MSP)	yes, next context
Frame	`ctx_t`, full NEON	`ctx64`, full q0–q31	hardware + software frame	`ucontext_t`
Interrupt stack	separate	separate	separate (MSP)	signal on thread stack

The table is the fastest way to see why the port contract exists. The kernel asks for six things — a core id, an IRQ mask, an atomic word, a recursive lock, an IPI, and a context switch — and each architecture answers them differently. The scheduler above the contract is identical.

---

14. Porting to a new SMP board

14.1 Checklist

1. **State the board facts** in `board.cmake`: `UOS_BOARD_NCPU`, the IPI number, the RAM window (`PORT_RAM_BASE` / `PORT_RAM_END`), the greeting, the banner, and the semihosting trap. Do not copy another board's numbers.
2. **Supply the width-dependent header** `include/cmsis-plus/rtos/port/os-c-decls.h`: clock/stack/IRQ types, the `smp_klock_t` / `smp_tlock_t` structures, `SMP_NO_OWNER`, and the `OS_USE_SMP_SCHEDULER` selection.
3. **Supply the inline operations** in `os-inlines.h`: `port_cpu_id`, `interrupts::critical_section::enter/exit`, `scheduler::lock/unlock/locked`, `_smp_klock_raw_acquire/release`, `wait_for_interrupt`, and the high-resolution clock hooks.
4. **Supply the port's** `src/rtos/os-core.cpp`: `port::scheduler::start`, `locked`, `reschedule`, `switch_stacks`, `context::create`, and `port_smp_secondary_start`.
5. **Define the lock storage** (`_smp_klock`, `_smp_tlock`, `_port_ctx_pending[]`, `lock_state[]`, and `kernel_lock_state[]` if used) in a board file such as `port_sys.cpp`.
6. **Implement the switch** in assembly or with `setjmp` / `ucontext`: save the full register file, return the incoming stack pointer, and implement the deferred publish where the architecture needs it.
7. **Implement the IPI** (`port_smp_ipi`) and its ISR.
8. **Implement the core release** (`smp::start_secondary_cores`) and the per-core timer/IRQ bring-up (`port_sys_init`).
9. **Create one idle thread per secondary** in the application (or in a shared `test-smp-boot.cpp`) and register it in `os_idle_thread_core[]` before releasing the core.
10. **Run the tests** in section 15, starting with a single-core image and then with `OS_NCPU=2`.

14.2 The three questions a new port must answer

Most bring-up failures reduce to one of these:

- **Where is the saved context safe to publish?** If the switch runs on the outgoing thread's stack, you need deferred publish. If it runs on an interrupt stack, you can publish immediately.
- **What excludes the other cores?** A hardware spinlock, an `LDREX/STREX` word, or a host atomic. Masking interrupts is not enough.
- **How does a core learn that work is available?** A private tick is enough for bounded latency; an IPI is needed for a wake-up that cannot wait a tick.

If those three are right, the rest is board plumbing.

---

15. Testing what was built

15.1 What each class of test proves

Test	What it exercises
`smp_test0`	Single-core bring-up: UART, MMU, 1 ms tick, scheduler; cores 1..N parked. Does not exercise cross-core behaviour.
`smp_test1`	Cross-core semaphore ping-pong with explicit affinity; the logger prints the core each participant ran on
`smp_test2`	Lock coherency: one worker per core, a shared counter under a mutex, final count == `OS_NCPU * ITER`
`smp_test3`	Cross-core message queue, blocking send/receive, a producer and a consumer on different cores
`smp_test4`	No-affinity work distribution: 8 CPU-bound workers, per-worker histogram over cores; proves cores share runnable work
`smp-num-test`	Numeric workload across cores, bounded run time
`smp-pipeline-test`	Multi-stage pipeline across cores, queues and back-pressure
`smp-pro-cons-test`	Producer/consumer with counting semaphores and queues
`smp-mat-test`, `smp-mat-sdcard-test`	Matrix workload, SD/FatFs from several cores
`smp-list-storage-test`, `smp-shapes-test`	Object/list and shape workloads; stress the allocator and the ready list
`mutex-stress`, `rtos-apis`, `cmsis-os-validator`	API conformance and stress, including SMP thread control
`clock-test`	Ticks, high-resolution clock, sleep accuracy across cores

The same shared test sources are compiled by every port that can run them. The test must not name an instruction or a register; what it needs from the machine it takes through the port headers (`port_cpu_id`, `port::scheduler:: wait_for_interrupt`, `timer_arm::get_count`, `PORT_BANNER_*`).

15.2 Running them

From `micro-os-plus-iii/tests`:

```
xpm run test-smp-cmake            # the eight QEMU-capable SMP configurations
xpm run test-aarch64-rpi4b-cmake  # one board's group
xpm run test-2xcortex-m33-cmake   # QEMU dual Cortex-M33
xpm run test-cortexm-pico2-cmake  # RP2350 board/QEMU group
```

One configuration, step by step:

```
xpm run prepare --config aarch64-rpi4b-cmake-gcc-debug
xpm run build   --config aarch64-rpi4b-cmake-gcc-debug
xpm run test    --config aarch64-rpi4b-cmake-gcc-debug

cd build/aarch64-rpi4b-cmake-gcc-debug
ctest -N                      # list the tests
ctest -R '^aarch64-rpi4b-smp_test2-qemu$' --output-on-failure
```

A hardware run uses the same build plus the board's `hw.sh` (OpenOCD, a CMSIS-DAP probe, a serial console). The test's verdict is the `RESULT: PASS` line and/or the CTest exit code; a run that never prints it is a timeout, not a pass.

15.3 What a healthy boot looks like

A typical AArch64 board prints, in order:

```
Scheduler: µOS++ Raspberry Pi 4 B (BCM2711, 4x Cortex-A72 SMP, ...)
Scheduler frequency: 1000 ticks/sec
...
os_main on core 0
Releasing cores 1..3...
join: c1=3 c2=3 c3=3 (Nms)
...
[worker on each core] ...
RESULT: PASS
```

The `join:` line is the first useful diagnostic. A missing core, a core that never ticked, or an idle thread that was not registered shows up there before any workload runs.

16. Limits, costs and sharp edges

No design is free. These are the deliberate limits of this one.

1. **Static core count.** `OS_NCPU` is a compile-time constant and the arrays are sized from it. There is no CPU hot-plug; a core that never starts simply never runs anything, and a thread pinned to it never runs.
2. **One global ready list and one kernel lock.** Every scheduling decision, every list insertion, every object operation takes the same lock. On two to four cores this is cheap and simple; on many cores it is a scalability ceiling. There are no per-CPU run queues to reduce contention.

3. **No remote preemption except at a tick or an IPI.** A higher-priority thread made ready by another core can wait up to one tick (1 ms by default) unless the kernel sends an IPI. The kernel sends one only for an affinity mismatch on resume and for `kill()`.
4. **Idle stealing, not load balancing.** A running thread is never migrated to another core. An idle core takes the highest-priority ready thread it is allowed to run. That distributes work when threads block or are created, but it does not equalise CPU load between busy cores.
5. **Affinity masks are not validated.** `cpu_affinity(0)` produces a thread that can never be selected. The setter simply stores the mask. Do not compute a mask that excludes all cores.
6. **Secondary idle threads are application-supplied.** The kernel has one `os_idle` (CPU 0); each secondary needs a thread registered in `os_idle_thread_core[c]` before release. If it is missing, the picker has no fallback and the port halts.
7. **Reaping happens in the idle loop.** Exited threads are destroyed by `os_rtos_idle_actions()` on CPU 0. A system that never lets CPU 0 idle can delay reaping, and therefore `join()`, indefinitely.
8. **The application scheduler lock is port-dependent.** On AArch64 it only stops this core's switches; on AArch32/Cortex-M/POSIX it also takes the kernel spinlock and masks interrupts. Portable application code should not depend on either the cost or the cross-core exclusion of `scheduler::critical_section`; use a mutex for data shared between cores.
9. **The kernel lock is not a priority-inheritance lock.** It relies on the holder having interrupts masked, so the holder finishes the section. A long section still stalls every other core that needs the lock. Keep sections short and never put I/O in them.
10. **Only CPU 0 advances time.** If CPU 0 is stopped (debugger, a very long semihosting trap), the system clock stops even though the other cores keep executing. The timer catch-up moves `hr_base` to compensate, but elapsed wall time is not recoverable in real time.
11. **The timer leaf lock is reserved, not used.** `_smp_tlock` and `port_tmr_lock()` are part of the port contract but have no kernel caller in this tree. Do not assume the timer queue is protected by a separate lock; it is protected by the interrupts/kernel section.
12. **SMP is a build variant, not a runtime mode.** You cannot switch a running image from scalar to SMP. `OS_NCPU` and `OS_USE_SMP_SCHEDULER` are compiled in, and the port's context-switch code differs.

---

17. Source map and glossary

17.1 Where to look

Question	File
Scheduler core, picker, per-CPU current thread	`micro-os-plus-iii/src/rtos/os-core.cpp`
Scheduler declarations, kernel sections	`include/cmsis-plus/rtos/os-sched.h`
Thread object, states, affinity	`include/cmsis-plus/rtos/os-thread.h`, `src/rtos/os-thread.cpp`
Thread lists, wait lists, clock lists	`include/cmsis-plus/rtos/internal/os-lists.h`, `src/rtos/internal/os-lists.cpp`
Idle thread and reaper	`src/rtos/os-idle.cpp`
Clocks, tick handler, timers	`src/rtos/os-clocks.cpp`, `src/rtos/os-timer.cpp`
Mutex/condvar/semaphore/queue/flags	`src/rtos/os-mutex.cpp`, `os-condvar.cpp`, `os-semaphore.cpp`, `os-mqueue.cpp`, `os-evflags.cpp`
Heap and `new`	`src/libc/stdlib/malloc.cpp`, `src/libcpp/new.cpp`
C API	`src/rtos/os-c-wrapper.cpp`
Shared port declarations	`port/smp-common/cmsis-plus/rtos/port/os-decls.h`
AArch32 port	`micro-os-plus-iii-aarch32/include/cmsis-plus/rtos/port/*`, `src/rtos/os-core.cpp`, `src/handlers.cpp`, `src/context_switch.cpp`
AArch64 port	`micro-os-plus-iii-aarch64/include/cmsis-plus/rtos/port/*`, `src/rtos/os-core.cpp`, `boards/rpi4b/src/startup.S`
Cortex-M port	`micro-os-plus-iii-cortexm/include-*`, `src/rtos/os-core-*.cpp`
POSIX host port	`micro-os-plus-iii-posix-arch/src/rtos/os-core.cpp`, `src/host_cpu.cpp`
Board facts and contract	`test/boards/<board>/board.cmake`, `test_smpl/src/board-contract.cpp`
Shared SMP test scaffolding	`test/<board>/src/test-smp-boot.cpp`

17.2 Glossary

- **CPU** / **core** — one hardware execution unit with its own registers, stack pointer and interrupt state. In this document they are interchangeable.

- **Port** — the architecture-specific half of the kernel: instructions, types, context switch, boot glue.
- **Board** — the memory map, peripherals, interrupt routing and release mechanism for one product.
- **Kernel lock** — the recursive spinlock (`_smp_klock`) plus the local interrupt mask. It protects all shared kernel data.
- **Scheduler lock** — the application-visible lock that stops this core from switching threads. Its cross-core strength depends on the port.
- **Kernel section** — `scheduler::kernel_critical_section`; the kernel's own internal lock, distinct from the application's scheduler lock on AArch64.
- **Claim** — setting a thread's `stack_ptr` to `nullptr` before returning into it, so no other core can pick it.
- **Publish** — storing a complete saved frame pointer into `stack_ptr` after the CPU has left the stack.
- **IPI** — inter-processor interrupt; used to request a reschedule on another core.
- **Idle thread** — the thread a core runs when nothing else is ready. CPU 0 uses the kernel's `os_idle`; secondaries use application-provided threads.
- **Affinity** — the bitmask of CPUs on which a thread may run.
- **Tick** — the periodic timer interrupt that drives preemption and, on CPU 0, the system clock.

17.3 Related documents

- `docs/smp-integration/smp-construction.md` — the repository layout and the port contract from the migration's point of view.
- `docs/smp-integration/MICRO-OS-PLUS-SMP-VS-SINGLECORE-ANALYSIS.md` — a broader analysis of the single-core versus SMP design, with additional code-level detail.
- `docs/STATUS.md` — the measured state of every board and test, including the bugs that produced several of the rules above.
- `docs/tests/` — the test harness, board guides and command references.

18. What happens when...?

A quick, plain-language reference for the situations that come up most often.

Situation	What the kernel does
A thread calls `sleep_for(10)`	Removes it from the ready list, puts a timeout node on the clock list, and switches to another thread. Ten ticks later the timer removes the node and resumes the thread.
A thread waits on a mutex already held	Suspends it on the mutex's wait list. The owner's `unlock()` wakes one waiter. The woken thread runs on the unlocker's core if allowed and higher priority, otherwise on another core at its tick or after an IPI.
Two threads print without a mutex	Their bytes interleave. The kernel does not protect the UART; the device is application data. Use a µOS++ mutex around the console.
A thread is woken on another core	It goes back on the ready list. The target core notices at its next tick, or earlier if the kernel sends an IPI because of affinity.
A thread exits	It joins the terminated list and switches away. The idle thread later frees its stack, marks it destroyed, and wakes a joiner.
A thread is killed while running on another core	The killer marks it suspended under the kernel lock, sends an IPI to that core, and waits until the thread is off every core and its context is saved.
A core has nothing to run	It runs its idle thread, which waits for an interrupt (`wfi` on ARM, `sigsuspend` on POSIX) and then checks the ready list again.
A secondary core is released	It enters `port_smp_secondary_start`, sets up its own timer and IPI, installs a fake current thread, and reschedules into its idle or first real thread.
A secondary core never prints anything	Check the release handshake, the cache clean before the release, the idle-thread registration, `port_sys_init()`, and `_port_core_alive[core]`.
A high-priority thread is made ready	On the current core it preempts immediately. On another core it waits for that core's tick, unless an IPI is sent.
A thread takes `scheduler::critical_section`	This core stops switching threads. On AArch64 other cores keep running; on AArch32, Cortex-M and POSIX the same section also holds the kernel spinlock and blocks them.
Two cores call `malloc` or `new`	The kernel lock serialises them. There is one heap, not one per core.
A running thread changes its affinity	The new mask is stored immediately. The thread keeps running until it blocks or is switched out; the next selection only considers cores in the mask.
A thread is created with affinity `0`	It can never be selected. Always leave at least one bit set.

19. Common misunderstandings

Question	Short answer
Does each core have its own scheduler?	No. There is one scheduler and one ready list. Each core has its own "current thread" pointer.
Is disabling interrupts enough to protect shared data?	Only on one core. The other cores are not affected. Use the kernel lock or a mutex.
Does `scheduler::critical_section` protect data shared between cores?	Not on AArch64, where it is only a per-core switch lock. On AArch32, Cortex-M and POSIX it also takes the kernel spinlock, but portable code should not rely on that difference.
Can two cores run the same thread?	No. The claim flag (`stack_ptr == nullptr`) marks a thread whose context is live on a core, and the picker skips it.
Will a higher-priority thread preempt another core immediately?	Usually not. The target core notices at its next tick, or earlier if the kernel sends an IPI.
Is the heap per core?	No. One heap, protected by the kernel lock.
Does `resume()` always run the woken thread immediately?	No. It runs immediately only if this core is allowed to run it and it has higher priority; otherwise it waits on the ready list, or an IPI is sent to a core that is allowed.
Is the idle thread special?	It is a normal thread at the lowest priority, one per core. The kernel's `os_idle` is CPU 0's; secondaries get application-provided idle threads.
Is the kernel lock a mutex?	No. It is a spinlock with interrupts masked. It never blocks, and it has no priority inheritance. Keep sections short.
Why is `main` always on core 0?	`os_main` pins it there, and the CMSIS-RTOS v1 compatibility layer pins the threads it creates there, because the v1 API assumes one core.
Does `yield()` make another core run?	No. It reschedules the current core. The other cores are already running or idle independently.
With default affinity, is a thread load-balanced?	It may run on any core, but a thread that is already running is never migrated. Idle cores take ready threads from the shared list; busy cores keep theirs.

20. Debug by symptom

Symptom	Most likely cause	Where to look
Every core spins forever, no output	Lock release order wrong, or a core died holding the lock	`_smp_klock.depth` / `owner` must be cleared before `lock = 0`; see 5.4
Random crash or wild jump inside a thread	A thread was resumed before its context was fully saved	Deferred publish and the `stack_ptr` claim; see 7.2
Corruption only when three or four cores run	Application data protected by the scheduler lock instead of a mutex, or a missing barrier	5.3, 5.4 and 11
`join()` never returns	The reaper never ran because CPU 0 never idled, or the target is still current on a core	`os_rtos_idle_actions()`, `current_thread_[c]`; see 12
A high-priority thread runs late	Remote preemption is tick-bounded; no IPI was sent	6.3 and 9
A secondary core never prints	Release handshake failed, cache not cleaned, idle thread not registered	8.2 and 8.3
`no ready thread and no idle thread on Core N`	`idleN` was not created or not stored in `os_idle_thread_core[]` before release	6.2 and 8.1
The clock stops or drifts	CPU 0 stalled; or skipped ticks were not accounted for in the high-resolution origin	10
A thread never runs at all	Affinity mask is zero or names a core that was never released	6.2 and 16
An IPI seems to be lost	The receiver's IPI source is disabled, the wrong SGI/mailbox is used, or interrupts stayed masked too long	9 and the board's `port_sys.cpp`

21. One-page cheat sheet

The five invariants

1. One ready list and one recursive kernel spinlock protect all shared kernel data.
2. One `current_thread_[cpu]` per core; a running thread is not in the ready list.
3. A thread is claimable only when its saved context is published; a core clears the flag when it claims the thread.
4. The kernel lock is not the application scheduler lock. On AArch64 the scheduler lock is local to one core.

5. Never switch while owning the kernel lock. Defer the switch, or refuse it.

The three locks

Lock	Protects	Scope
`interrupts::critical_section`	Lists, queues, shared fields	IRQ mask + kernel spinlock
`scheduler::critical_section`	This core's thread switches	Per-core on AArch64; also the kernel lock on the other ports
`scheduler::kernel_critical_section`	Heap, `new`, `atexit`, object internals	IRQ mask + kernel spinlock, plus a per-core flag on AArch64

Boot a secondary in six steps

1. Core 0 creates and registers `idleN` in `os_idle_thread_core[N]`.
2. Core 0 writes the entry address to the board's release mechanism.
3. The secondary enters `port_smp_secondary_start`.
4. It calls `port_sys_init()`: its timer, its IPI, its interrupt interface.
5. It installs a fake current thread and calls `reschedule()`.
6. It runs its idle thread or the first ready thread, and sets `_port_core_alive[N]` on its first tick.

Thread states

`ready` → `running` → (`suspended` → `ready`) or (`terminated` → `destroying` → `destroyed`).

When an IPI is sent

- a woken thread is not allowed on the waking core (affinity mismatch);
- a thread running on another core is being killed.

Everything else waits for the target core's next tick: at most 1 ms by default.

Numbers to remember

- `OS_NCPU`: the number of cores, fixed at build time.
- 1 ms: the default tick, and the worst-case remote preemption delay.
- `0`: the value of a claimed `stack_ptr`; never resume a thread with it.
- `SMP_NO_OWNER`: the value of `_smp_klock.owner` when the lock is free.

22. Read the source in this order

If you want to see the implementation, these six functions tell most of the story. Read them in this order; each one is short.

1. `scheduler::internal_switch_threads()` in `src/rtos/os-core.cpp`. This is the decision: re-link the outgoing thread if it is still runnable, scan the ready list for the first thread allowed on this CPU, select it and remove it from the list, or fall back to idle.
2. `port::scheduler::switch_stacks()` in the port's `src/rtos/os-core.cpp`. This is the switch: take the kernel lock, stage the outgoing frame, clear the incoming claim flag, release the lock, return the incoming stack pointer. Read the comment about the release order.
3. `port_isr_handler()` in the board's `src/rtos/port_isr.cpp`. This is how a tick or an IPI becomes a switch: acknowledge, set the pending flag, and switch on the way out if the core is not locked.
4. `port_smp_secondary_start()` in the port's `src/rtos/os-core.cpp`. This is how another core joins: mask, set up its timer and IPI, install a fake current thread, reschedule.
5. `thread::resume()` in `src/rtos/os-thread.cpp`. This is how a blocked thread becomes runnable: link the ready node under the lock, reschedule, and send an IPI if the thread is pinned elsewhere.
6. `port::scheduler::kernel_locked()` in the AArch64 port. This is the modern kernel lock: an interrupt critical section plus a per-core flag, with a deferred switch when the section ends. Comparing it with the AArch32 `locked()` shows why the two ports differ.

After those six, the port's `os-inlines.h` is the machine-specific half of the story: core id, IRQ mask, atomic acquire/release, and the spinlock word.